

El Algoritmo de Grover

El problema de búsqueda

Sea L una lista **no** ordenada con K elementos

- $K = 2^n$
- Cada elemento de L tiene b bits
- Los valores n y b pueden ser distintos

Dado $a \in \{0, 1\}^b$, queremos encontrar un índice $i \in \{0, 1\}^n$ tal que $L[i] = a$

Complejidad clásica del problema de búsqueda

¿Cuántas posiciones de L es necesario consultar en un algoritmo clásico?

- No suponemos que la lista está ordenada

En el peor caso necesitamos realizar K consultas

- ¿Cómo se demuestra esto?

Complejidad clásica del problema de búsqueda

¿Cuántas posiciones de L es necesario consultar en un algoritmo clásico?

- No suponemos que la lista está ordenada

Un algoritmo clásico que soluciona el problema realiza entonces $\Omega(K)$ operaciones

- Vamos a reducir este orden utilizando un algoritmo cuántico

Predicado que reconoce el valor a

Definimos $m_a : \{0, 1\}^b \longrightarrow \{0, 1\}$ como:

$$m_a(x) = \begin{cases} 1 & x = a \\ 0 & x \neq a \end{cases}$$

Definimos $f_a : \{0, 1\}^n \longrightarrow \{0, 1\}$ como $f_a(i) = m_a(L[i])$

Por lo tanto: $f_a(i) = 1$ si y sólo si $L[i] = a$

- El algoritmo tiene que encontrar entonces un índice i tal que $f_a(i) = 1$

La noción de qRAM

Una **qRAM** contiene una memoria interna con celdas

La celda con dirección i almacena $L[i]$:

dirección	contenido
0	$L[0]$
1	$L[1]$
$\vdots$	$\vdots$
$K - 1$	$L[K - 1]$

La lista permanece almacenada en esta memoria durante la ejecución del algoritmo

Registros de una consulta

Una consulta utiliza dos registros cuánticos:

- A : registro de direcciones con n qubits
- R : registro de datos con b qubits

El registro A indica la celda que queremos consultar, mientras que el registro R recibe el contenido de esa celda

- La QPU prepara ambos registros y los conecta con la interfaz de la qRAM

Acceso proporcionado por la qRAM

La qRAM proporciona una operación unitaria U_L definida como

$$U_L (|d\rangle \otimes |i\rangle) = |d \oplus L[i]\rangle \otimes |i\rangle$$

La dirección consultada es $|i\rangle$, la cual es colocada en el registro A

El contenido inicial del registro R es $|d\rangle$

- Recibe el resultado de la consulta en la forma $|d \oplus L[i]\rangle$

Acceso proporcionado por la qRAM

La qRAM proporciona una operación unitaria U_L definida como

$$U_L (|d\rangle \otimes |i\rangle) = |d \oplus L[i]\rangle \otimes |i\rangle$$

Utilizamos U_L como una caja negra

- Cada aplicación cuenta como una consulta
- Cada una de estas consultas tiene costo 1

Lectura de elementos

Para leer el elemento almacenado en la posición i , inicializamos R en cero:

$$U_L (|0^b\rangle \otimes |i\rangle) = |L[i]\rangle \otimes |i\rangle$$

Además, la QPU puede preparar $\sum_{i=0}^{K-1} \alpha_i |i\rangle$, y realizar la siguiente consulta:

$$U_L \left(|0^b\rangle \otimes \sum_{i=0}^{K-1} \alpha_i |i\rangle \right) = \sum_{i=0}^{K-1} \alpha_i (|L[i]\rangle \otimes |i\rangle)$$

Lectura de elementos

Además, la QPU puede preparar $\sum_{i=0}^{K-1} \alpha_i |i\rangle$, y realizar la siguiente consulta:

$$U_L \left(|0^b\rangle \otimes \sum_{i=0}^{K-1} \alpha_i |i\rangle \right) = \sum_{i=0}^{K-1} \alpha_i (|L[i]\rangle \otimes |i\rangle)$$

La consulta conserva la coherencia entre las direcciones

- Esta consulta tiene costo 1

Información obtenida mediante una consulta

Después de aplicar U_L , los registros R y A pueden quedar entrelazados:

$$\sum_{i=0}^{K-1} \alpha_i (|L[i]\rangle \otimes |i\rangle)$$

Una medición entrega un par $(L[i], i)$ con probabilidad $|\alpha_i|^2$

Alcance de la qRAM

La qRAM proporciona:

- La memoria interna que almacena L
- El mecanismo de direccionamiento coherente
- La interfaz para los registros A y R
- La operación reversible U_L

La qRAM no compara $L[i]$ con a , no decide si el índice i es una solución y no amplifica probabilidades

- Estas operaciones las realiza un algoritmo implementado en una QPU

Uso de la qRAM

Suponemos que la lista L está cargada en memoria

La QPU ejecuta el circuito del algoritmo de búsqueda

- Aplica U_L cada vez que necesita consultar la lista

La misma memoria se reutiliza durante todas las iteraciones

- La qRAM puede estar integrada en la QPU o conectada como un subsistema coherente
- La ubicación física no modifica la definición de U_L

Oráculo XOR para m_a

En el capítulo anterior vimos cómo construir un oráculo XOR U_{m_a} para m_a

- U_{m_a} tiene $O(b)$ compuertas y utiliza r qubits auxiliares, donde $r \leq b$
- U_{m_a} tiene que ser construido por el algoritmo de búsqueda, lo cual toma tiempo $O(b)$
- U_{m_a} tiene que ser ejecutado por el algoritmo de búsqueda, lo cual también toma tiempo $O(b)$

U_{m_a} realiza la siguiente transformación para $x \in \{0, 1\}^b$ e $y \in \{0, 1\}$:

$$U_{m_a} (|y\rangle \otimes |0^r\rangle \otimes |x\rangle) = |y \oplus m_a(x)\rangle \otimes |0^r\rangle \otimes |x\rangle$$

Oráculo XOR para f_a

Combinamos U_L con U_{m_a} para generar un oráculo XOR U_{f_a} para f_a :

$$U_{f_a} = (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(I^{\otimes(1+r)} \otimes U_L)$$

Tenemos que:

- U_{f_a} realiza dos llamadas al oráculo U_L
- U_{f_a} tiene $O(b)$ compuertas y utiliza $r + b$ qubits auxiliares, donde $r \leq b$
- U_{f_a} tiene que ser construido por el algoritmo de búsqueda, lo cual toma tiempo $O(b)$
- U_{f_a} tiene que ser ejecutado por el algoritmo de búsqueda, lo cual también toma tiempo $O(b)$

Oráculo XOR para f_a

U_{f_a} realiza la siguiente transformación para $i \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$U_{f_a} (|y\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle) = |y \oplus f_a(i)\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle$$

De hecho, tenemos que:

$$\begin{aligned} U_{f_a} (|y\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle) &= (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(I^{\otimes(1+r)} \otimes U_L)(|y\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle) \\ &= (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n}) \left(I^{\otimes(1+r)} (|y\rangle \otimes |0^r\rangle) \otimes U_L(|0^b\rangle \otimes |i\rangle) \right) \\ &= (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(|y\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle) \end{aligned}$$

Oráculo XOR para f_a

$$\begin{aligned} U_{f_a}(|y\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle) \\ &= (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(|y\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle) \\ &= (I^{\otimes(1+r)} \otimes U_L) (U_{m_a}(|y\rangle \otimes |0^r\rangle \otimes |L[i]\rangle) \otimes I^{\otimes n}|i\rangle) \\ &= (I^{\otimes(1+r)} \otimes U_L)(|y \oplus m_a(L[i])\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle) \\ &= (I^{\otimes(1+r)} \otimes U_L)(|y \oplus f_a(i)\rangle \otimes |0^r\rangle \otimes |L[i]\rangle \otimes |i\rangle) \\ &= I^{\otimes(1+r)}(|y \oplus f_a(i)\rangle \otimes |0^r\rangle) \otimes U_L(|L[i]\rangle \otimes |i\rangle) \\ &= |y \oplus f_a(i)\rangle \otimes |0^r\rangle \otimes |L[i] \oplus L[i]\rangle \otimes |i\rangle \\ &= |y \oplus f_a(i)\rangle \otimes |0^{r+b}\rangle \otimes |i\rangle \end{aligned}$$

Oráculo de fase para la búsqueda

Usando lo aprendido en la sección anterior, construimos el oráculo de fase O_{f_a} para f_a a partir de U_{f_a} :

$$O_{f_a} = (X \otimes I^{\otimes(r+b+n)})(H \otimes I^{\otimes(r+b+n)})U_{f_a}(H \otimes I^{\otimes(r+b+n)})(X \otimes I^{\otimes(r+b+n)})$$

Vale decir:

$$\begin{aligned} O_{f_a} = & (X \otimes I^{\otimes(r+b+n)})(H \otimes I^{\otimes(r+b+n)}) \\ & (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(I^{\otimes(1+r)} \otimes U_L) \\ & (H \otimes I^{\otimes(r+b+n)})(X \otimes I^{\otimes(r+b+n)}) \end{aligned}$$

Oráculo de fase para la búsqueda

Vale decir:

$$\begin{aligned} O_{f_a} = & (X \otimes I^{\otimes(r+b+n)})(H \otimes I^{\otimes(r+b+n)}) \\ & (I^{\otimes(1+r)} \otimes U_L)(U_{m_a} \otimes I^{\otimes n})(I^{\otimes(1+r)} \otimes U_L) \\ & (H \otimes I^{\otimes(r+b+n)})(X \otimes I^{\otimes(r+b+n)}) \end{aligned}$$

Para $x \in \{0, 1\}^n$, tenemos que: $O_{f_a} |0^{1+r+b}x\rangle = (-1)^{f_a(x)} |0^{1+r+b}x\rangle$

Oráculo de fase para la búsqueda

Tenemos que:

- O_{f_a} realiza dos llamadas al oráculo U_L
- O_{f_a} tiene $O(b)$ compuertas y utiliza $1 + r + b$ qubits auxiliares, donde $r \leq b$
- O_{f_a} tiene que ser construido por el algoritmo de búsqueda, lo cual toma tiempo $O(b)$
- O_{f_a} tiene que ser ejecutado por el algoritmo de búsqueda, lo cual también toma tiempo $O(b)$

¿Es posible usar una qRAM en los computadores cuánticos de IBM?

No, los computadores cuánticos de IBM no ofrecen una qRAM

- No existe una qRAM persistente separada a la que el circuito pueda hacer consultas coherentes

Para ejecutar un algoritmo como el de Grover en los computadores cuánticos de IBM, los oráculos deben construirse como circuitos

- Vamos a ver dos consecuencias fundamentales del algoritmo de Grover en el caso en que el oráculo es dado como circuito

La presentación del algoritmo de Grover

Sea $M = |\{i \mid L[i] = a\}|$

Presentamos el algoritmo de Grover bajo los siguientes supuestos:

1. Suponemos que $M \geq 1$, M es conocido de antemano y O_{f_a} no usa qubits auxiliares
 - $O_{f_a}|x\rangle = (-1)^{f_a(x)}|x\rangle$ para $x \in \{0, 1\}^n$
2. Permitimos que O_{f_a} use qubits auxiliares
 - $O_{f_a}|0^{1+r+b}x\rangle = (-1)^{f_a(x)}|0^{1+r+b}x\rangle$ para $x \in \{0, 1\}^n$
3. No suponemos que $M \geq 1$ ni que M es conocido de antemano

Superposición inicial

El circuito para el algoritmo de Grover tiene n qubits

- Cada $i \in \{0, 1\}^n$ representa una posición de L , dado que L tiene $K = 2^n$ elementos
- La entrada del circuito es $|0^n\rangle$

Definimos:
$$|s\rangle = H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{K}} \sum_{i=0}^{K-1} |i\rangle$$

Cada índice tiene inicialmente probabilidad $\frac{1}{K}$

Estados buenos y malos

Considere las siguientes combinaciones de buenos y malos estados:

$$|G\rangle = \frac{1}{\sqrt{M}} \sum_{i \in \{0, \dots, K-1\} : L[i]=a} |i\rangle$$

$$|B\rangle = \frac{1}{\sqrt{K-M}} \sum_{i \in \{0, \dots, K-1\} : L[i] \neq a} |i\rangle$$

Vamos a ver más adelante que podemos suponer que $M < K$

- Por lo tanto, la definición de $|B\rangle$ es correcta

Tenemos que $\langle G|B\rangle = 0$

Plano de Grover

El estado $|s\rangle$ puede escribirse como: $|s\rangle = \sqrt{\frac{M}{K}}|G\rangle + \sqrt{\frac{K-M}{K}}|B\rangle$

Puesto que:

$$\begin{aligned} & \sqrt{\frac{M}{K}}|G\rangle + \sqrt{\frac{K-M}{K}}|B\rangle \\ &= \sqrt{\frac{M}{K}} \frac{1}{\sqrt{M}} \sum_{i \in \{0, \dots, K-1\} : L[i]=a} |i\rangle + \sqrt{\frac{K-M}{K}} \frac{1}{\sqrt{K-M}} \sum_{i \in \{0, \dots, K-1\} : L[i] \neq a} |i\rangle \\ &= \frac{1}{\sqrt{K}} \sum_{i \in \{0, \dots, K-1\} : L[i]=a} |i\rangle + \frac{1}{\sqrt{K}} \sum_{i \in \{0, \dots, K-1\} : L[i] \neq a} |i\rangle = \frac{1}{\sqrt{K}} \sum_{i=0}^{K-1} |i\rangle = |s\rangle \end{aligned}$$

Plano de Grover

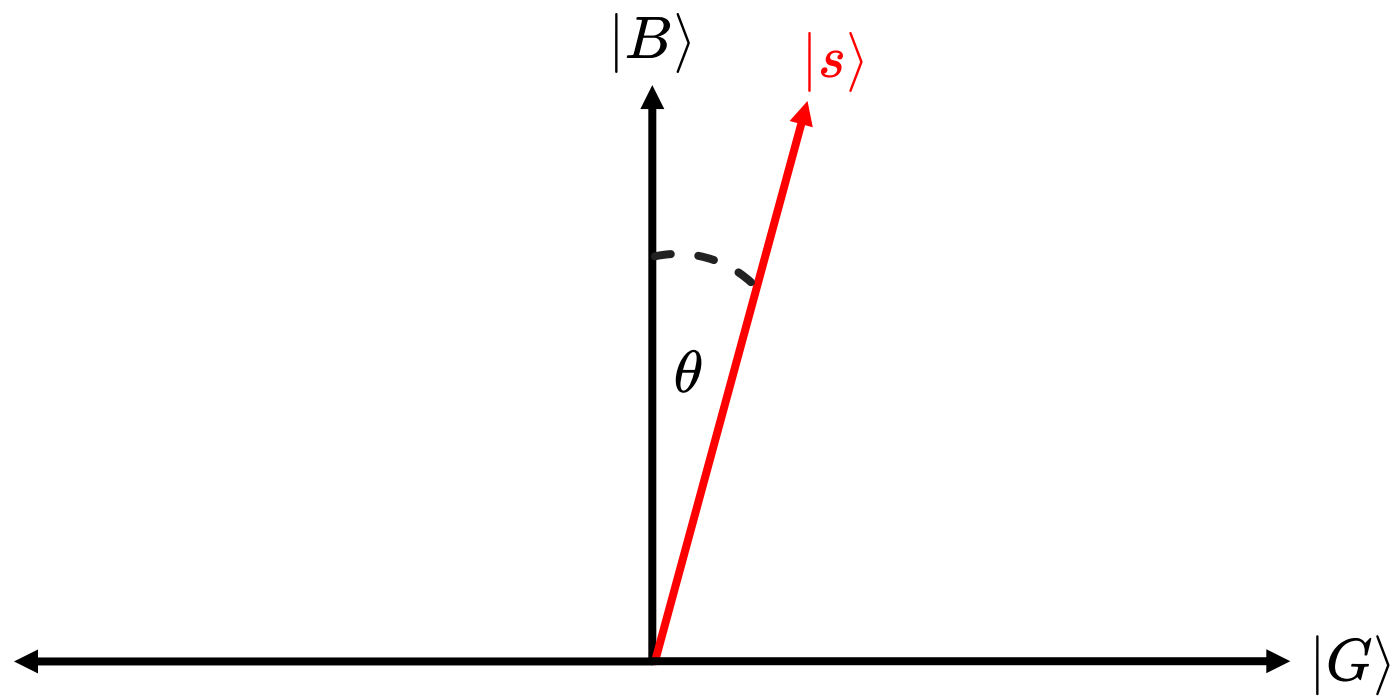
El estado $|s\rangle$ puede escribirse como: $|s\rangle = \sqrt{\frac{M}{K}}|G\rangle + \sqrt{\frac{K-M}{K}}|B\rangle$

Sea $\theta \in \left[0, \frac{\pi}{2}\right]$ tal que: $\sin \theta = \sqrt{\frac{M}{K}}$ $\cos \theta = \sqrt{\frac{K-M}{K}}$

El estado $|s\rangle$ puede entonces escribirse como:

$$|s\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle$$

Plano de Grover



En el plano definido por $|G\rangle$ y $|B\rangle$

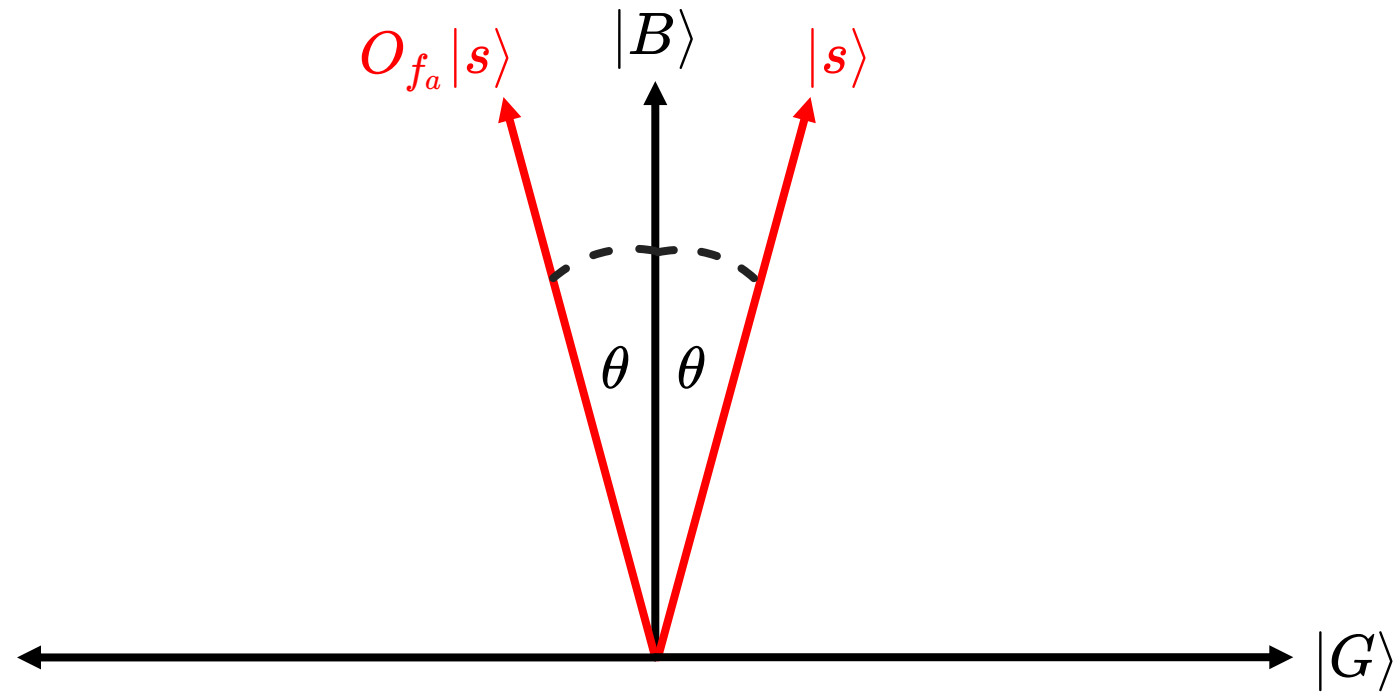
Reflexión producida por el oráculo

Tenemos que $O_{f_a}|G\rangle = -|G\rangle$ y $O_{f_a}|B\rangle = |B\rangle$

Por lo tanto $O_{f_a}|s\rangle = -\sin(\theta)|G\rangle + \cos(\theta)|B\rangle$

- El oráculo realiza una reflexión respecto del eje $|B\rangle$

Reflexión producida por el oráculo



En el plano definido por $|G\rangle$ y $|B\rangle$

Operador de difusión

Definimos el siguiente operador: $D = 2|s\rangle\langle s| - I^{\otimes n}$

- Vamos a ver más adelante cómo se implementa este operador

Dado $|\psi\rangle = \sum_{i=0}^{K-1} \alpha_i |i\rangle$, vamos a demostrar que $D|\psi\rangle = \sum_{i=0}^{K-1} (2\bar{\alpha} - \alpha_i) |i\rangle$

- Donde $\bar{\alpha} = \frac{1}{K} \sum_{i=0}^{K-1} \alpha_i$

Operador de difusión

Tenemos que:

$$\langle s | \psi \rangle = \left(\frac{1}{\sqrt{K}} \sum_{i=0}^{K-1} \langle i | \right) \left(\sum_{j=0}^{K-1} \alpha_j |j\rangle \right) = \frac{1}{\sqrt{K}} \sum_{i=0}^{K-1} \alpha_i = \frac{\sqrt{K}}{K} \sum_{i=0}^{K-1} \alpha_i = \sqrt{K\bar{\alpha}}$$

Por lo tanto:

$$\begin{aligned} D|\psi\rangle &= (2|s\rangle\langle s| - I^{\otimes n})|\psi\rangle = 2|s\rangle\langle s | \psi \rangle - I^{\otimes n}|\psi\rangle = 2\sqrt{K\bar{\alpha}}|s\rangle - |\psi\rangle \\ &= 2\sqrt{K\bar{\alpha}} \left(\frac{1}{\sqrt{K}} \sum_{i=0}^{K-1} |i\rangle \right) - \left(\sum_{i=0}^{K-1} \alpha_i |i\rangle \right) = \sum_{i=0}^{K-1} (2\bar{\alpha} - \alpha_i) |i\rangle \end{aligned}$$

¿Por qué D se llama operador de difusión?

D es llamado operador de difusión porque cada nueva amplitud depende del promedio de todas las amplitudes

- Vale decir, la acción de D mezcla información global sobre todas las amplitudes

La difusión como reflexión

Para cada $|\psi\rangle \in \mathbb{C}^{2^n}$, se tiene que $|\psi\rangle = \beta|s\rangle + |\psi_\perp\rangle$, donde:

- $\beta = \langle s | \psi \rangle$
- $|\psi_\perp\rangle = |\psi\rangle - \beta|s\rangle$

Se tiene que $\langle s | \psi_\perp \rangle = 0$

- $|s\rangle$ y $|\psi_\perp\rangle$ son vectores ortogonales

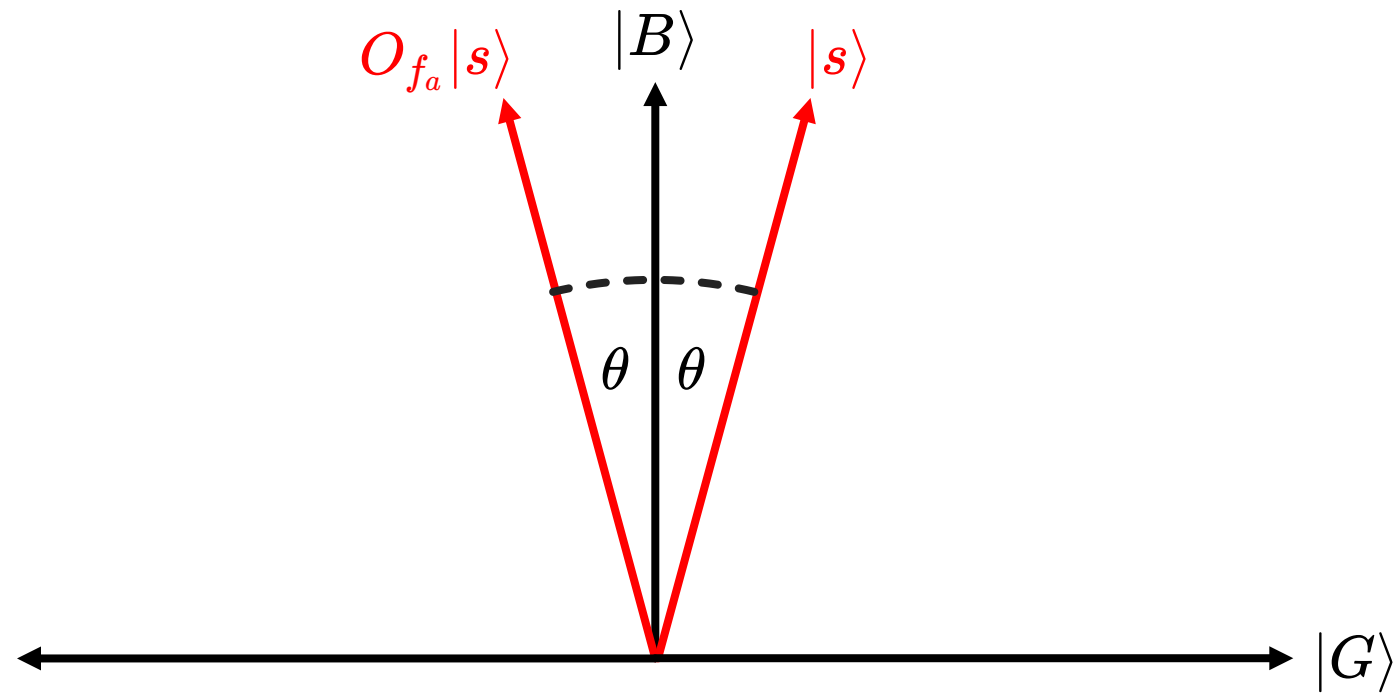
La difusión como reflexión

Además, tenemos que:

$$\begin{aligned} D|\psi\rangle &= (2|s\rangle\langle s| - I^{\otimes n})|\psi\rangle = 2|s\rangle\langle s|\psi\rangle - |\psi\rangle \\ &= 2\beta|s\rangle - |\psi\rangle = \beta|s\rangle + (\beta|s\rangle - |\psi\rangle) = \beta|s\rangle - |\psi_{\perp}\rangle \end{aligned}$$

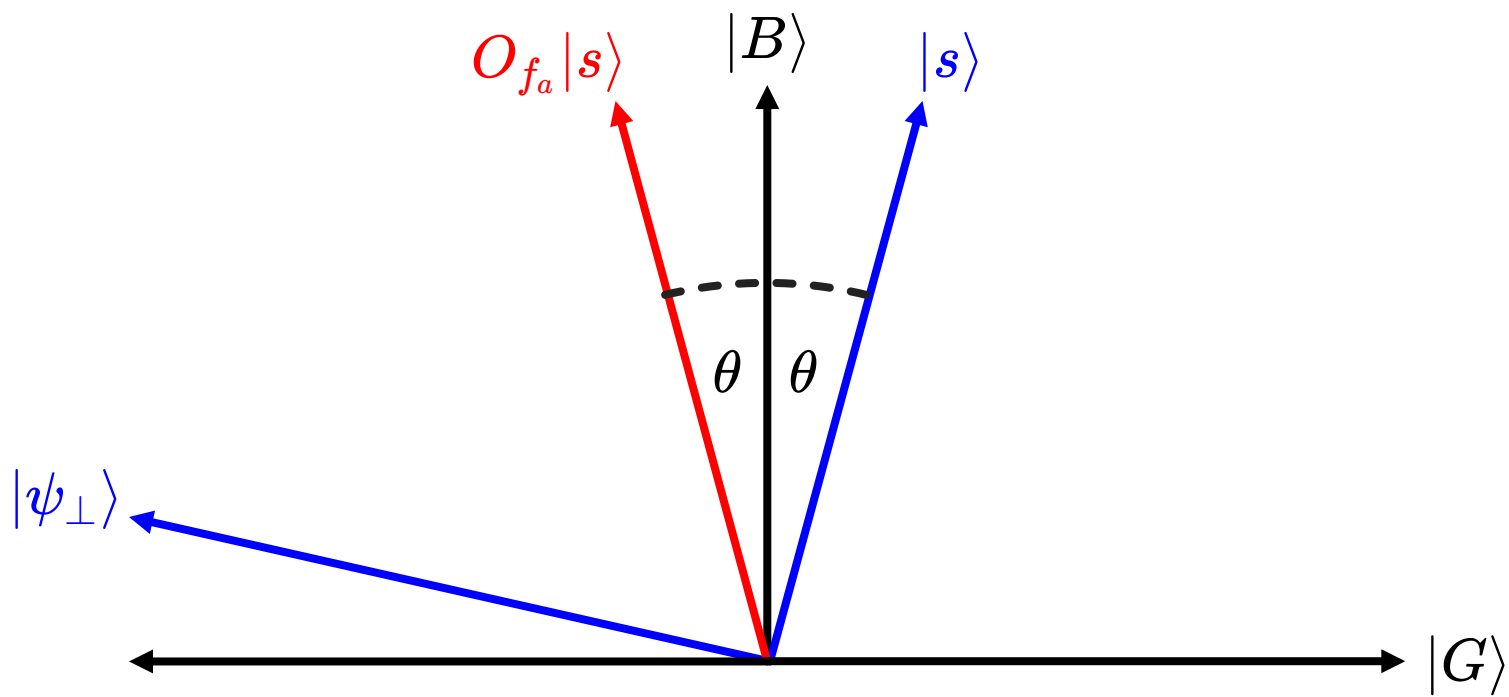
D realiza una reflexión respecto del eje $|s\rangle$

El resultado de ambas operaciones



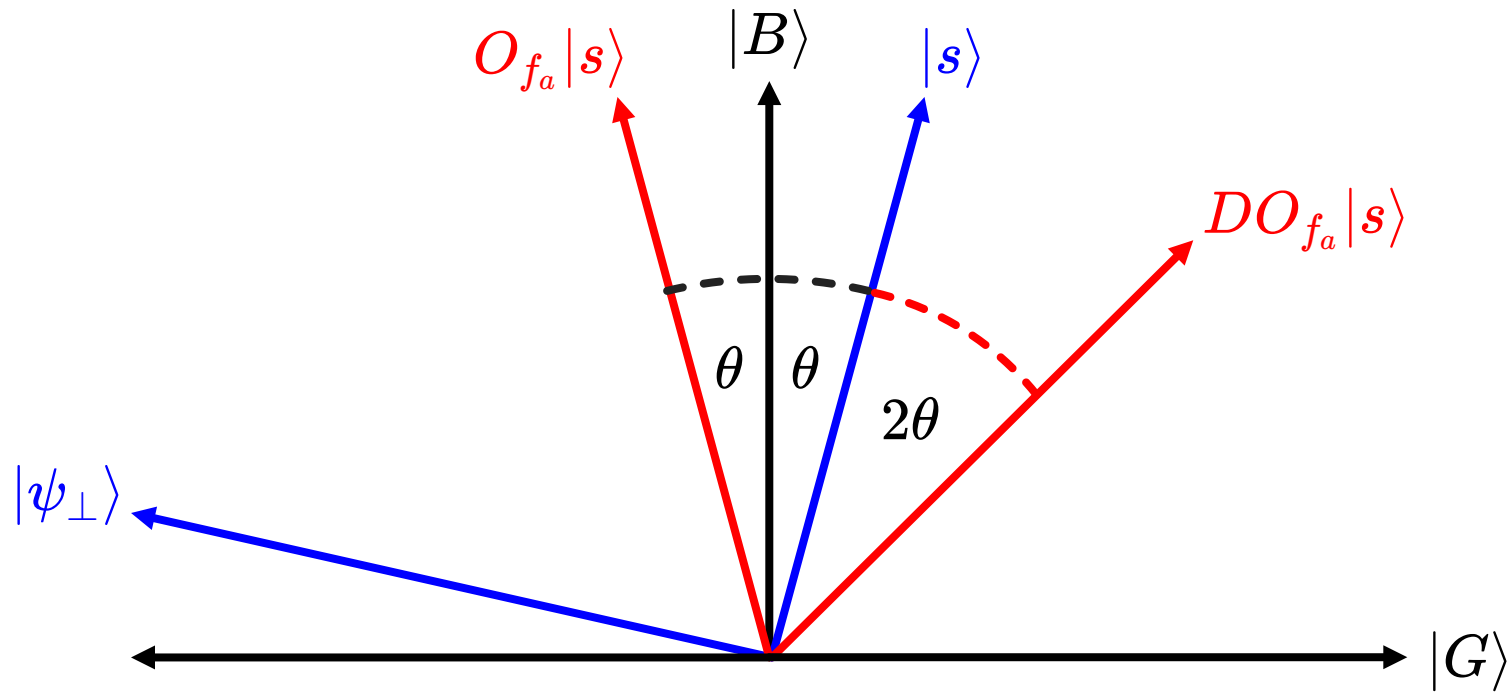
En el plano definido por $|G\rangle$ y $|B\rangle$

El resultado de ambas operaciones



En el plano definido por $|G\rangle$ y $|B\rangle$

El resultado de ambas operaciones



En el plano definido por $|G\rangle$ y $|B\rangle$

El resultado de ambas operaciones

Vamos a deducir la transformación anterior de manera algebraica

Recuerde que:

$$|s\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle$$
$$O_{f_a}|s\rangle = -\sin(\theta)|G\rangle + \cos(\theta)|B\rangle$$

Por lo tanto:

$$\begin{aligned}\langle s|O_{f_a}|s\rangle &= (\sin(\theta)\langle G| + \cos(\theta)\langle B|)(-\sin(\theta)|G\rangle + \cos(\theta)|B\rangle) \\ &= -\sin^2(\theta) + \cos^2(\theta) = \cos(2\theta)\end{aligned}$$

El resultado de ambas operaciones

De esto deducimos que:

$$\begin{aligned} DO_{f_a}|s\rangle &= (2|s\rangle\langle s| - I^{\otimes n})O_{f_a}|s\rangle \\ &= 2|s\rangle\langle s|O_{f_a}|s\rangle - I^{\otimes n}O_{f_a}|s\rangle \\ &= 2\cos(2\theta)|s\rangle - O_{f_a}|s\rangle \\ &= 2\cos(2\theta)(\sin(\theta)|G\rangle + \cos(\theta)|B\rangle) - (-\sin(\theta)|G\rangle + \cos(\theta)|B\rangle) \\ &= (2\cos(2\theta)\sin(\theta) + \sin(\theta))|G\rangle + (2\cos(2\theta)\cos(\theta) - \cos(\theta))|B\rangle \end{aligned}$$

El resultado de ambas operaciones

Y tenemos que:

$$\begin{aligned}\sin(3\theta) &= \sin(\theta + 2\theta) \\ &= \sin(\theta) \cos(2\theta) + \cos(\theta) \sin(2\theta) \\ &= \sin(\theta) \cos(2\theta) + \cos(\theta) \sin(\theta + \theta) \\ &= \sin(\theta) \cos(2\theta) + \cos(\theta)(2 \sin(\theta) \cos(\theta)) \\ &= \sin(\theta) \cos(2\theta) + \sin(\theta)(2 \cos^2(\theta)) \\ &= \sin(\theta) \cos(2\theta) + \sin(\theta)((2 \cos^2(\theta) - 1) + 1) \\ &= \sin(\theta) \cos(2\theta) + \sin(\theta)(\cos(2\theta) + 1) \\ &= 2 \cos(2\theta) \sin(\theta) + \sin(\theta)\end{aligned}$$

El resultado de ambas operaciones

Y además tenemos que:

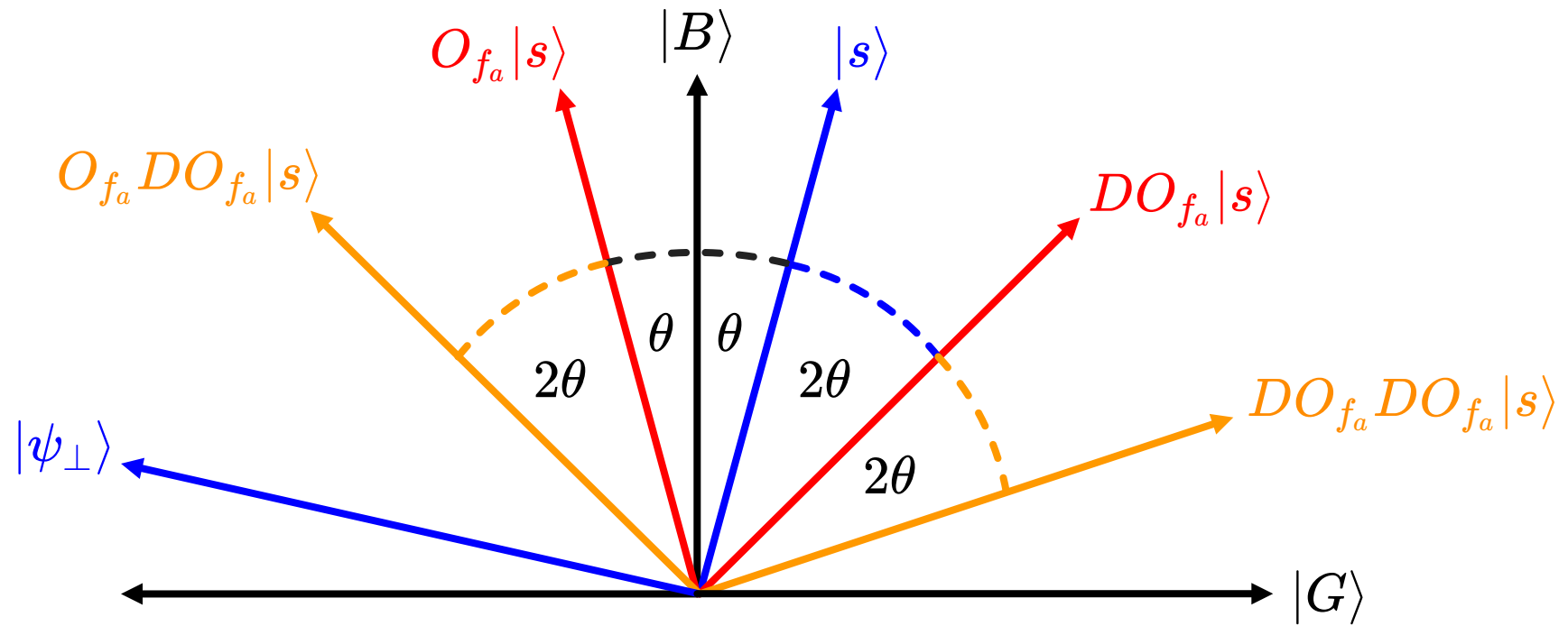
$$\begin{aligned}\cos(3\theta) &= \cos(\theta + 2\theta) \\ &= \cos(\theta) \cos(2\theta) - \sin(\theta) \sin(2\theta) \\ &= \cos(\theta) \cos(2\theta) - \sin(\theta) \sin(\theta + \theta) \\ &= \cos(\theta) \cos(2\theta) - \sin(\theta)(2 \sin(\theta) \cos(\theta)) \\ &= \cos(\theta) \cos(2\theta) - \cos(\theta)(2 \sin^2(\theta)) \\ &= \cos(\theta) \cos(2\theta) - \cos(\theta)(2(1 - \cos^2(\theta))) \\ &= \cos(\theta) \cos(2\theta) - \cos(\theta)(1 + (1 - 2 \cos^2(\theta))) \\ &= \cos(\theta) \cos(2\theta) - \cos(\theta)(1 - \cos(2\theta)) \\ &= 2 \cos(2\theta) \cos(\theta) - \cos(\theta)\end{aligned}$$

El resultado de ambas operaciones

Concluimos que:

$$\begin{aligned} DO_{f_a}|s\rangle &= (2\cos(2\theta)\sin(\theta) + \sin(\theta))|G\rangle + (2\cos(2\theta)\cos(\theta) - \cos(\theta))|B\rangle \\ &= \sin(3\theta)|G\rangle + \cos(3\theta)|B\rangle \end{aligned}$$

¿Y si aplicamos DO_{f_a} dos veces?



En el plano definido por $|G\rangle$ y $|B\rangle$

Las iteraciones del algoritmo

En cada iteración, el algoritmo de Grover aplica DO_{f_a}

El estado inicial $|s\rangle$ forma un ángulo θ con $|B\rangle$, y cada iteración agrega 2θ en la dirección de $|G\rangle$

Se puede deducir de manera algebraica que después de t iteraciones:

$$(DO_{f_a})^t |s\rangle = \sin((2t + 1)\theta) |G\rangle + \cos((2t + 1)\theta) |B\rangle$$

Las iteraciones del algoritmo

Después de t iteraciones, la probabilidad de medir uno de los índices i tal que $L[i] = a$ es:

$$\sin^2((2t + 1)\theta)$$

donde $\sin(\theta) = \sqrt{\frac{M}{K}}$

Esta probabilidad es 1 cuando $(2t + 1)\theta = \frac{\pi}{2}$

- Vale decir, cuando $t = \frac{\pi}{4\theta} - \frac{1}{2}$

Las iteraciones del algoritmo

El problema es que $t = \frac{\pi}{4\theta} - \frac{1}{2}$ puede no ser un número entero

Tenemos entonces que usar una aproximación de t , lo cual puede generar un error

- Vamos a acotar el valor de t , y a partir de esto vamos a acotar la probabilidad de error

Las iteraciones del algoritmo

Necesitamos acotar el ángulo θ para acotar el valor de t

Como $x \leq \arcsin(x)$ para $x \in [0, 1]$ y $\theta = \arcsin\left(\sqrt{\frac{M}{K}}\right)$, tenemos que:

$$\sqrt{\frac{M}{K}} \leq \theta$$

Considerando que $t = \frac{\pi}{4\theta} - \frac{1}{2}$, obtenemos que : $t \leq \frac{\pi}{4} \sqrt{\frac{K}{M}} - \frac{1}{2}$

Las iteraciones del algoritmo

Definimos $t_{\text{opt}} = \lfloor t \rfloor$

- t_{opt} es el entero más cercano a t
- Tenemos que $|t_{\text{opt}} - t| \leq \frac{1}{2}$

El algoritmo de Grover realiza t_{opt} iteraciones

- Tenemos entonces que realiza $O\left(\sqrt{\frac{K}{M}}\right)$ iteraciones

La probabilidad de encontrar una solución

¿Cómo se puede equivocar el algoritmo de Grover?

- El algoritmo se equivoca si para el índice i medido se tiene que $L[i] \neq a$

Una vez que el algoritmo mide i , verificamos de manera clásica si $L[i] = a$

- Si es así, el algoritmo retorna i
- En caso contrario, el algoritmo indica que falló

¿Cuál es la probabilidad de que el algoritmo falle?

La probabilidad de encontrar una solución

$$\text{Sea } \delta = (2t_{\text{opt}} + 1)\theta - \frac{\pi}{2}$$

Tenemos que la probabilidad de éxito es $\sin^2\left(\frac{\pi}{2} + \delta\right)$

Sabemos que $\sin\left(\frac{\pi}{2} + \delta\right) = \cos(\delta)$

- Por lo tanto, la probabilidad de éxito es $\cos^2(\delta)$

La probabilidad de encontrar una solución

La probabilidad de fallar es entonces $1 - \cos^2(\delta) = \sin^2(\delta)$

Tenemos que:

$$\delta = (2t_{\text{opt}} + 1)\theta - \frac{\pi}{2} = (2t_{\text{opt}} + 1)\theta - (2t + 1)\theta = 2\theta(t_{\text{opt}} - t)$$

Por lo tanto: $|\delta| = 2\theta|t_{\text{opt}} - t| \leq 2\theta\frac{1}{2} = \theta$

La probabilidad de encontrar una solución

Considerando que $0 \leq |\delta| \leq \theta \leq \frac{\pi}{2}$, concluimos que:

$$\sin^2(\delta) = \sin^2(|\delta|) \leq \sin^2(\theta) = \frac{M}{K}$$

Por lo tanto, la probabilidad de fallar es menor o igual a $\frac{M}{K}$

- ¿Qué tan grande es esta probabilidad?

La probabilidad de encontrar una solución

Vamos a suponer que $M \leq \frac{K}{2}$

- En general se tiene que $M \ll K$
- La probabilidad de fallar es entonces menor o igual a $\frac{1}{2}$

Además, la probabilidad de fallar es menor o igual a $\left(\frac{M}{K}\right)^r$ si repetimos el algoritmo r veces

- ¿Qué es lo que cambia entre distintas ejecuciones del algoritmo?

¿Qué hacemos si $M > \frac{K}{2}$?

Ejercicio: Suponiendo que $M > \frac{K}{2}$, dé un algoritmo aleatorizado de tiempo $O(b)$ que encuentra i tal que $L[i] = a$ con una probabilidad mayor o igual a $1 - \frac{1}{2^{100}}$

- Si retorna i entonces se sabe que $L[i] = a$
- Existe una probabilidad de no retornar un índice, la cual es menor o igual a $\frac{1}{2^{100}}$

Implementación del operador de difusión

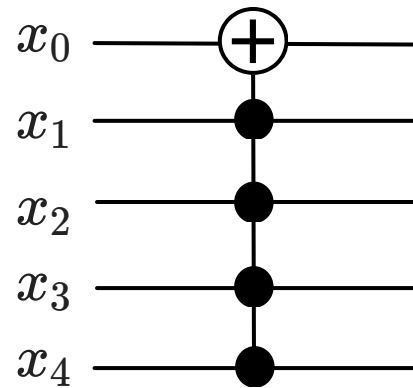
El último ingrediente que nos queda del algoritmo de Grover es indicar cómo se implementa $D = 2|s\rangle\langle s| - I^{\otimes n}$

Para esto tenemos que mostrar cómo implementar $D_0 = 2|0^n\rangle\langle 0^n| - I^{\otimes n}$

- Con esto resolvemos nuestro problema puesto que $D = H^{\otimes n} D_0 H^{\otimes n}$

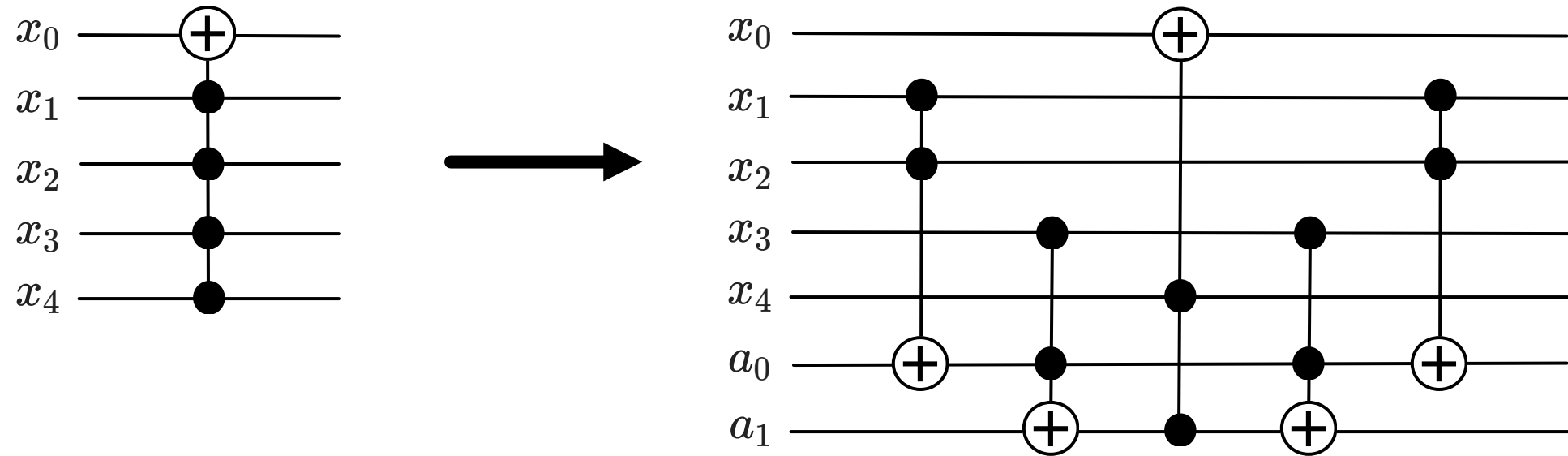
Implementación del operador de difusión

Considere primero esta compuerta:



Implementación del operador de difusión

Considere primero esta compuerta:

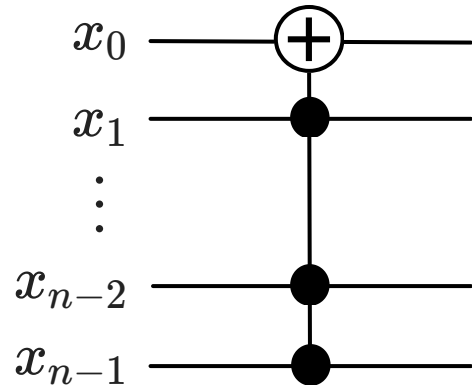


Esta compuerta transforma $|0^2 x_4 x_3 x_2 x_1 x_0\rangle$ en

$$|0^2 x_4 x_3 x_2 x_1 (x_0 \oplus (x_4 \wedge x_3 \wedge x_2 \wedge x_1))\rangle$$

Implementación del operador de difusión

Considere ahora la generalización de la compuerta anterior:



Implementamos esta compuerta de manera análoga a la compuerta anterior

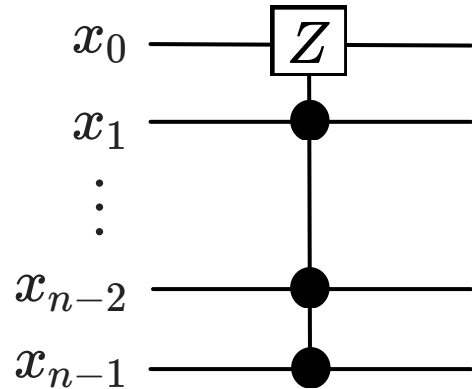
- Para $n \geq 3$, el circuito resultante tiene $2(n - 3) + 1$ compuertas

Esta compuerta transforma $|0^{n-3}x_{n-1}x_{n-2} \cdots x_1x_0\rangle$ en

$$|0^{n-3}x_{n-1}x_{n-2} \cdots x_1(x_0 \oplus (x_{n-1} \wedge x_{n-2} \wedge \cdots \wedge x_1))\rangle$$

Implementación del operador de difusión

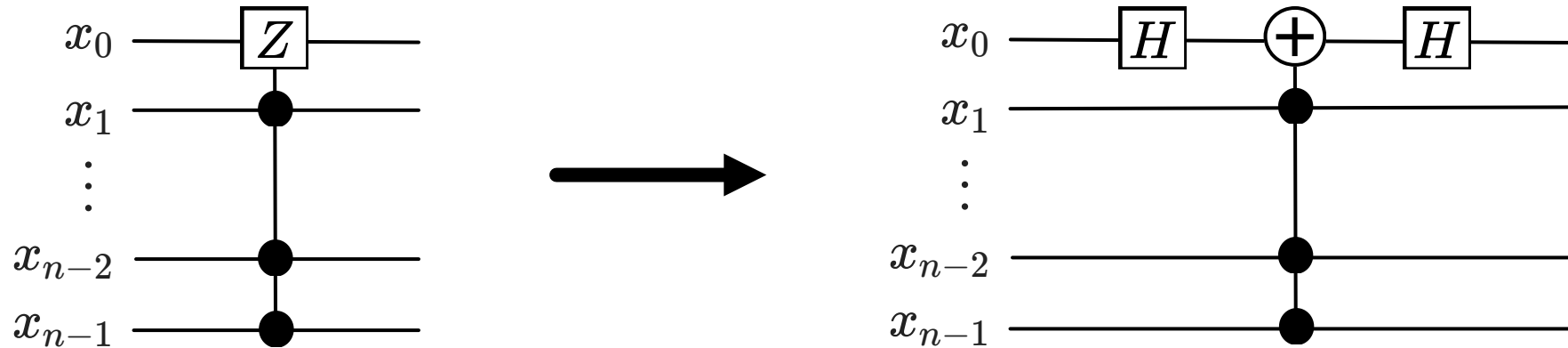
Considere ahora esta compuerta:



Recuerde que $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Implementación del operador de difusión

Considere ahora esta compuerta:

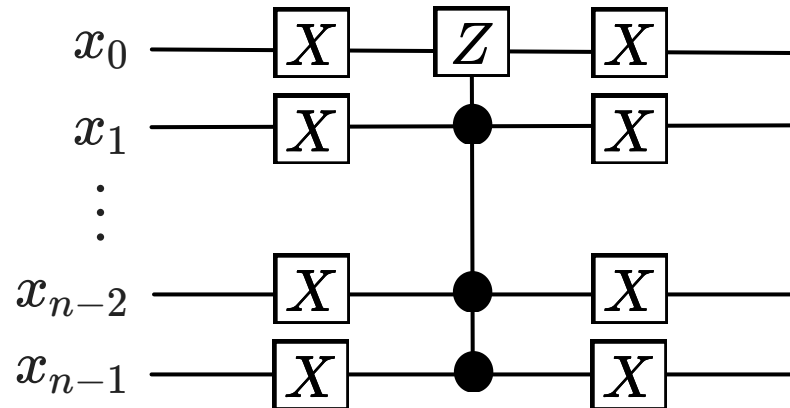


Recuerde que $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Tenemos que $HXH = Z$

Implementación del operador de difusión

Finalmente considere este circuito:



Este circuito realiza las siguientes transformaciones:

$$|0^n\rangle \longrightarrow -|0^n\rangle$$

$$|10^{n-1}\rangle \longrightarrow |10^{n-1}\rangle$$

$$|yx\rangle \longrightarrow |yx\rangle$$

$$y \in \{0, 1\}, x \in \{0, 1\}^{n-1}, x \neq 0^{n-1}$$

Implementación del operador de difusión

¿Por qué nos interesa este circuito?

Tenemos que:

$$D_0|0^n\rangle = (2|0^n\rangle\langle 0^n| - I^{\otimes n})|0^n\rangle = 2|0^n\rangle\langle 0^n | 0^n\rangle - I^{\otimes n}|0^n\rangle = 2|0^n\rangle - |0^n\rangle = |0^n\rangle$$

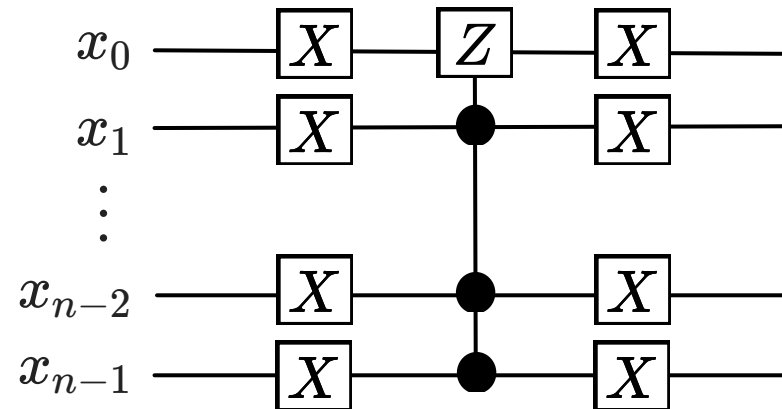
Para $x \in \{0, 1\}^n$ con $x \neq 0^n$:

$$D_0|x\rangle = (2|0^n\rangle\langle 0^n| - I^{\otimes n})|x\rangle = 2|0^n\rangle\langle 0^n | x\rangle - I^{\otimes n}|x\rangle = -|x\rangle$$

Implementación del operador de difusión

Por lo tanto, el siguiente circuito implementa $-D_0$

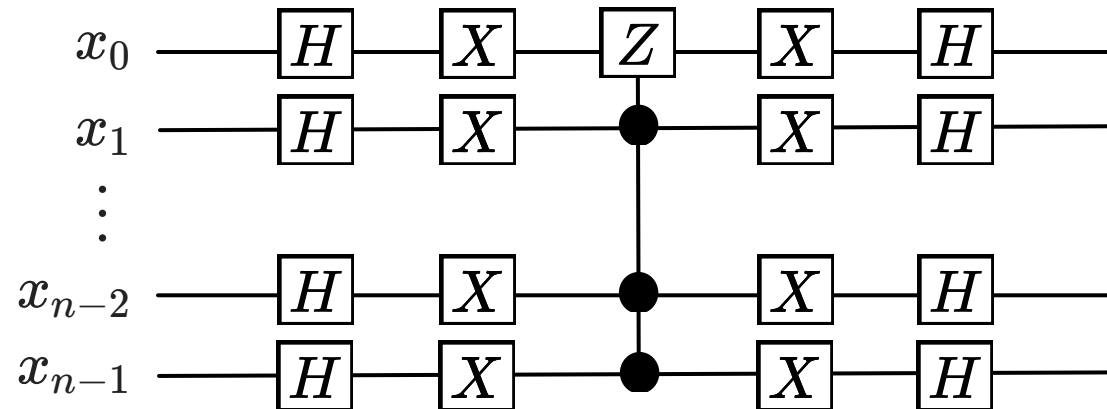
- Vale decir, el circuito implementa D_0 excepto por una fase global que podemos descartar



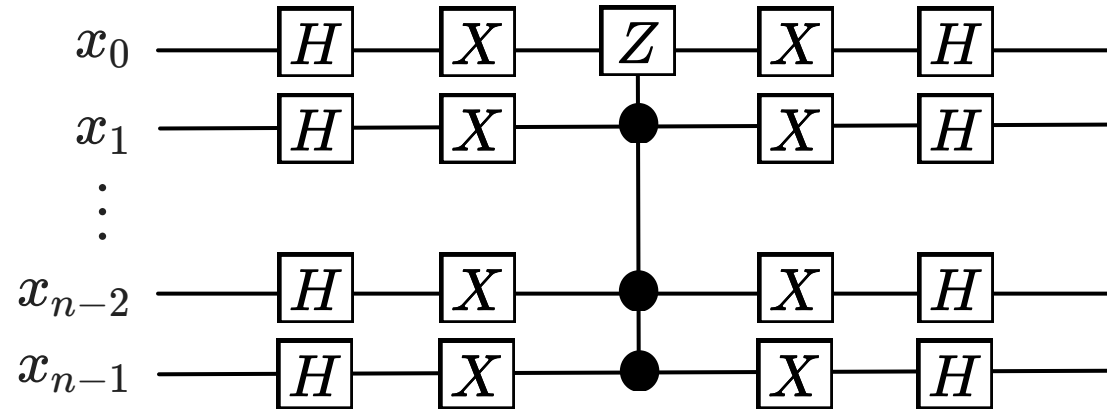
Implementación del operador de difusión

Como $D = H^{\otimes n} D_0 H^{\otimes n}$, el siguiente circuito implementa $-D$

- Vale decir, el circuito implementa D excepto por una fase global que podemos descartar



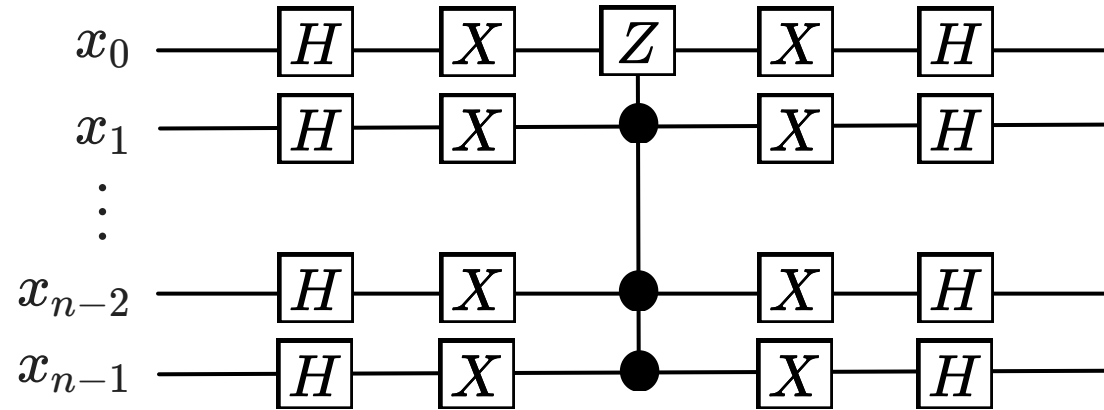
Implementación del operador de difusión



Este circuito tiene $O(n)$ compuertas, y puede ser ejecutado en tiempo $O(n)$

- En el algoritmo de Grover hay que considerar tanto el tiempo para construir como el tiempo para ejecutar D
- Recuerde que $n = \log_2(K)$

Implementación del operador de difusión



Este circuito tiene qubits auxiliares que no representamos en la figura

- Es simple manejar estos qubits auxiliares, que antes y después de ser utilizados tienen valor $|0\rangle$
- Por esto también es simple remover la restricción de que O_{f_a} no usa qubits auxiliares

El algoritmo de Grover

Tenemos todos los ingredientes necesarios para presentar el algoritmo de Grover

- La entrada del algoritmo es una lista L con $K = 2^n$ elementos, y un elemento $a \in \{0, 1\}^b$
 - Cada elemento de L tiene b bits
- La salida es un índice $i \in \{0, 1\}^n$ tal que $L[i] = a$, o **none**
- Suponemos que $M = |\{i \mid L[i] = a\}|$ es conocido, y además que
$$1 \leq M \leq \frac{K}{2}$$

El algoritmo de Grover

1. Sea $\theta = \arcsin \left(\sqrt{\frac{M}{K}} \right)$ y $t_{\text{opt}} = \left\lfloor \frac{\pi}{4\theta} - \frac{1}{2} \right\rfloor$
2. Construya el circuito $(DO_{f_a})^{t_{\text{opt}}} H^{\otimes n}$
3. **for** $j = 1$ **to** 100 **do**
 1. Ejecute $(DO_{f_a})^{t_{\text{opt}}} H^{\otimes n} |0^n\rangle$ y mida
 2. Sea $i \in \{0, 1\}^n$ el resultado de la medición
 3. **if** $L[i] = a$ **return** i
4. **return none**

El tiempo de ejecución y la probabilidad de fallar del algoritmo de Grover

Construir el circuito $(DO_{f_a})^{t_{\text{opt}}} H^{\otimes n}$ toma tiempo $O(\sqrt{K}(\log_2(K) + b))$

- Construir D y $H^{\otimes n}$ toma tiempo $O(n) = O(\log_2(K))$
- Construir O_{f_a} toma tiempo $O(b)$
- $t_{\text{opt}} \leq \frac{\pi}{4} \sqrt{\frac{K}{M}} \leq \frac{\pi}{4} \sqrt{K}$

El tiempo de ejecución y la probabilidad de fallar del algoritmo de Grover

Ejecutar $(DO_{f_a})^{t_{\text{opt}}} H^{\otimes n} |0^n\rangle$ toma tiempo $O(\sqrt{K}(\log_2(K) + b))$

- Ejecutar $H^{\otimes n}$ toma tiempo $O(1)$
- Ejecutar O_{f_a} toma tiempo $O(b)$
- Ejecutar D toma tiempo $O(n) = O(\log_2(K))$

Las comparaciones $L[i] = a$ toman tiempo $O(b)$

- Realizamos a lo más 100 de estas comparaciones

El tiempo de ejecución y la probabilidad de fallar del algoritmo de Grover

El tiempo de ejecución del algoritmo es $O(\sqrt{K}(\log_2(K) + b))$

La probabilidad de que el algoritmo falle es menor o igual a $\left(\frac{M}{K}\right)^{100}$

- Esta probabilidad es menor o igual a $\left(\frac{1}{2}\right)^{100}$ puesto que $M \leq \frac{K}{2}$

Eliminando los supuestos

Ejercicio: Modifique el algoritmo para permitir que O_{f_a} use qubits auxiliares

- Observe que ya permitimos usar qubits auxiliares en la implementación del operador de difusión D

¿Cómo se pueden eliminar los supuestos de que M es conocido y $M \geq 1$?

- ¿Por qué eliminar el supuesto de que M es conocido no es trivial?
- En el resto de la presentación vamos a eliminar estos supuestos

Eliminando el supuesto de que M es conocido

Eliminamos el supuesto de que M es conocido

- Mantenemos el supuesto de que $M \geq 1$, el cual vamos a eliminar al final de la presentación
- Y además mantenemos el supuesto de que $M \leq \frac{K}{2}$
 - Ya vimos cómo lidiar con el caso en que $M > \frac{K}{2}$

El ángulo real

En la formulación anterior del algoritmo usamos M para calcular:

$$\theta = \arcsin \left(\sqrt{\frac{M}{K}} \right)$$

Si no conocemos M no podemos calcular θ

El ángulo real

Aunque el algoritmo no conoce M , para analizarlo seguimos definiendo

$$\theta_M = \arcsin \left(\sqrt{\frac{M}{K}} \right)$$

Después de t iteraciones, la probabilidad de éxito es

$$p_t = \sin^2((2t + 1)\theta_M)$$

El menor ángulo posible

El menor ángulo posible ocurre cuando $M = 1$:

$$\theta_{\min} = \arcsin \left(\frac{1}{\sqrt{K}} \right)$$

Este valor sí lo conocemos, porque depende solamente de K

- No reemplazamos θ_M por $\theta_{\min}$, solo lo usamos en el análisis del nuevo algoritmo

Elegir t al azar

Sea $m = \lceil \sqrt{K} \rceil$

Elegimos t uniformemente al azar en $\{0, \dots, m - 1\}$

Y después ejecutamos $(DO_{f_a})^t H^{\otimes n} |0^n\rangle$

- Como en la versión anterior del algoritmo de Grover

Probabilidad de éxito

La probabilidad p_e de éxito es:

$$\begin{aligned}\Pr(\text{éxito}) &= \sum_{k=0}^{m-1} \Pr(\text{éxito} \mid t = k) \Pr(t = k) \\ &= \frac{1}{m} \sum_{k=0}^{m-1} \Pr(\text{éxito} \mid t = k) \\ &= \frac{1}{m} \sum_{t=0}^{m-1} \sin^2((2t + 1)\theta_M)\end{aligned}$$

Probabilidad promedio de éxito

Vamos a demostrar que:

$$p_e = \frac{1}{2} - \frac{\sin(4m\theta_M)}{4m \sin(2\theta_M)}$$

Dado que $\sin^2(x) = \frac{1 - \cos(2x)}{2}$, tenemos que:

$$p_e = \frac{1}{m} \sum_{t=0}^{m-1} \sin^2((2t+1)\theta_M) = \frac{1}{2} - \frac{1}{2m} \sum_{t=0}^{m-1} \cos((4t+2)\theta_M)$$

Probabilidad promedio de éxito

Sabemos que $2 \sin(x) \cos(y) = \sin(y + x) - \sin(y - x)$

Tomando $x = 2\theta_M$ e $y = (4t + 2)\theta_M$, concluimos que:

$$2 \sin(2\theta_M) \cos((4t + 2)\theta_M) = \sin((4t + 4)\theta_M) - \sin(4t\theta_M)$$

Por lo tanto:

$$2 \sin(2\theta_M) \sum_{t=0}^{m-1} \cos((4t + 2)\theta_M) = \sum_{t=0}^{m-1} [\sin((4t + 4)\theta_M) - \sin(4t\theta_M)]$$

Probabilidad promedio de éxito

Cancelando los términos intermedios obtenemos:

$$2 \sin(2\theta_M) \sum_{t=0}^{m-1} \cos((4t + 2)\theta_M) = \sin(4m\theta_M)$$

Concluimos que:

$$\sum_{t=0}^{m-1} \cos((4t + 2)\theta_M) = \frac{\sin(4m\theta_M)}{2 \sin(2\theta_M)}$$

Y de esto concluimos que: $p_e = \frac{1}{2} - \frac{\sin(4m\theta_M)}{4m \sin(2\theta_M)}$

Una cota inferior para p_e

Dado que $\sin(4m\theta_M) \leq 1$, obtenemos que:

$$p_e \geq \frac{1}{2} - \frac{1}{4m \sin(2\theta_M)}$$

$$\sin(2\theta_M) = 2 \sin(\theta_M) \cos(\theta_M) = 2\sqrt{\frac{M}{K}} \sqrt{1 - \frac{M}{K}}$$

Sabemos que:

$$\sin(2\theta_{\min}) = 2 \sin(\theta_{\min}) \cos(\theta_{\min}) = 2\sqrt{\frac{1}{K}} \sqrt{1 - \frac{1}{K}}$$

Una cota inferior para p_e

$$\sin(2\theta_M) = 2 \sin(\theta_M) \cos(\theta_M) = 2\sqrt{\frac{M}{K}} \sqrt{1 - \frac{M}{K}}$$

Sabemos que:

$$\sin(2\theta_{\min}) = 2 \sin(\theta_{\min}) \cos(\theta_{\min}) = 2\sqrt{\frac{1}{K}} \sqrt{1 - \frac{1}{K}}$$

Pero supusimos que $1 \leq M \leq \frac{K}{2}$, por lo que $\sin(2\theta_{\min}) \leq \sin(2\theta_M)$

Una cota inferior para p_e

Dado que $\sin(2\theta_{\min}) \leq \sin(2\theta_M)$, obtenemos que:

$$p_e \geq \frac{1}{2} - \frac{1}{4m \sin(2\theta_{\min})}$$

Como $m = \lceil \sqrt{K} \rceil$, tenemos que $\sqrt{K} \leq m$, y concluimos que:

$$\frac{1}{4m \sin(2\theta_{\min})} \leq \frac{1}{4\sqrt{K} \sin(2\theta_{\min})}$$

Una cota inferior para p_e

Dado que: $\sin(2\theta_{\min}) = 2 \sin(\theta_{\min}) \cos(\theta_{\min}) = 2\sqrt{\frac{1}{K}} \sqrt{1 - \frac{1}{K}}$

Concluimos que:

$$\frac{1}{4m \sin(2\theta_{\min})} \leq \frac{1}{4\sqrt{K} \sin(2\theta_{\min})} = \frac{1}{4\sqrt{K} 2\sqrt{\frac{1}{K}} \sqrt{1 - \frac{1}{K}}} = \frac{\sqrt{K}}{8\sqrt{K-1}} \leq \frac{1}{4}$$

Por lo tanto: $p_e \geq \frac{1}{2} - \frac{1}{4m \sin(2\theta_{\min})} \geq \frac{1}{4}$

¿Cuántas repeticiones tenemos que hacer?

Cada intento tiene probabilidad de éxito mayor o igual a $\frac{1}{4}$

Si repetimos r veces el algoritmo, la probabilidad de fallar todas las veces es menor o igual a $\left(\frac{3}{4}\right)^r$

- Por lo tanto, la probabilidad de éxito es mayor o igual a $1 - \left(\frac{3}{4}\right)^r$

Por ejemplo, con 100 intentos la probabilidad de éxito es mayor que 0.99999999999999

¿Qué hacemos si $M = 0$?

En la versión anterior del algoritmo de Grover calculamos

$$\theta = \arcsin \left(\sqrt{\frac{M}{K}} \right) \text{ y } t_{\text{opt}} = \left\lfloor \frac{\pi}{4\theta} - \frac{1}{2} \right\rfloor$$

- Si $M = 0$, tenemos que $\theta = 0$, y t_{opt} no está bien definido

En la nueva versión del algoritmo no vamos a tener este problema puesto que no calculamos θ , y t se elige al azar en el intervalo $\{0, \dots, \lceil \sqrt{K} \rceil - 1\}$

¿Qué hacemos si $M = 0$?

Además, después de generar un índice $i \in \{0, 1\}^n$, el algoritmo se pregunta si $L[i] = a$, y sólo si esta condición es cierta retorna i

Por lo tanto, si $M = 0$, entonces el algoritmo retorna **none**

- Lo cual es correcto puesto que no hay una posición $i \in \{0, 1\}^n$ tal que $L[i] = a$

El algoritmo (completo) de Grover

Tenemos todos los ingredientes necesarios para presentar el algoritmo de Grover

- La entrada del algoritmo es una lista L con $K = 2^n$ elementos, y un elemento $a \in \{0, 1\}^b$
 - Cada elemento de L tiene b bits
- La salida es un índice $i \in \{0, 1\}^n$ tal que $L[i] = a$, o **none**
- Suponemos que $M \leq \frac{K}{2}$

El algoritmo (completo) de Grover

1. **for** $j = 1$ **to** 100 **do**

1. Elija al azar con distribución uniforme $t \in \{0, \dots, \lceil \sqrt{K} \rceil - 1\}$
2. Construya el circuito $(DO_{f_a})^t H^{\otimes n}$
3. Ejecute $(DO_{f_a})^t H^{\otimes n} |0^n\rangle$ y mida
4. Sea $i \in \{0, 1\}^n$ el resultado de la medición
5. **if** $L[i] = a$ **return** i

2. **return none**

El tiempo de ejecución y la probabilidad de fallar del algoritmo (completo) de Grover

Del análisis para la primera versión del algoritmo concluimos que el tiempo de ejecución del algoritmo completo es $O(\sqrt{K}(\log_2(K) + b))$

La probabilidad de que el algoritmo falle es menor o igual a $\left(\frac{3}{4}\right)^{100}$

- Para la versión anterior del algoritmo esta probabilidad era menor o igual a $\left(\frac{1}{2}\right)^{100}$

Primera consecuencia: criptografía

El protocolo de criptografía simétrica más usado es AES

- Dos usuarios de AES deben compartir una clave secreta k para cifrar y descifrar mensajes
- La función de cifrado es Enc_k : dado un texto plano m , su texto cifrado es $c = Enc_k(m)$
- La función de descifrado es Dec_k : dado un texto cifrado c , su texto descifrado es $m = Dec_k(c)$
- Para cada mensaje m tenemos que $m = Dec_k(Enc_k(m))$

Primera consecuencia: criptografía

AES-128 usa una clave k y un mensaje m de 128 bits

- Se extiende a mensajes de largo arbitrario usando los modos de operación

También se utilizan las versiones AES-192 y AES-256

Primera consecuencia: criptografía

Un posible ataque a AES-128 es el siguiente: dado un texto plano m y su cifrado c , obtenga una clave k tal que $Enc_k(m) = c$

- ¿Cómo se puede obtener el par (m, c) ?

En un computador clásico, un algoritmo ingenuo realiza 2^{128} evaluaciones de AES-128 en el peor caso para encontrar k

- Todos los ataques conocidos en computadores clásicos tienen una complejidad similar
- Los ataques más rápidos realizan aproximadamente 2^{126} evaluaciones de AES-128

Primera consecuencia: criptografía

Podemos usar el algoritmo de Grover para reducir considerablemente el número de operaciones en un ataque a AES-128

Las funciones de cifrado y descifrado de AES-128 pueden representarse como circuitos

Existe un circuito clásico $C : \{0, 1\}^{128} \times \{0, 1\}^{128} \longrightarrow \{0, 1\}^{128}$ tal que para cada $k, m \in \{0, 1\}^{128}$: $C(k, m) = Enc_k(m)$

- ¿Dónde se utiliza un circuito como este?

Primera consecuencia: criptografía

Sea (m_0, c_0) un par que va a ser utilizado en un ataque a AES-128

- $m_0, c_0 \in \{0, 1\}^{128}$
- c_0 es el texto cifrado obtenido a partir del texto plano m_0

Construimos los siguientes circuitos clásicos:

- $D : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$ tal que $D(x) = C(x, m_0)$
- $E : \{0, 1\}^{128} \rightarrow \{0, 1\}$ tal que $E(x) = 1$ si y solo si $x = c_0$
- $F : \{0, 1\}^{128} \rightarrow \{0, 1\}$ tal que $F(x) = E(D(x))$

Primera consecuencia: criptografía

Para el circuito F construimos un circuito cuántico U_F tal que para $x \in \{0, 1\}^{128}$ e $y \in \{0, 1\}$:

$$U_F |y 0^r x\rangle = |(y \oplus F(x)) 0^r x\rangle$$

Si F tiene S compuertas, U_F utiliza $O(S)$ compuertas y $O(S)$ qubits auxiliares

Primera consecuencia: criptografía

Para $k \in \{0, 1\}^{128}$, tenemos que $F(k) = 1$ si y solo si $Enc_k(m_0) = c_0$

¿Qué representa entonces U_F ?

- Si suponemos que $k \in \{0, 1\}^{128}$ es una posición en una qRAM, ¿qué nos indica el oráculo U_F ?

Por supuesto no construimos esta qRAM, la tenemos dada de manera implícita a través del circuito U_F

Primera consecuencia: criptografía

Construimos O_F a partir de U_F , y utilizamos el algoritmo de Grover para el oráculo O_F

Podemos descubrir entonces una clave $k \in \{0, 1\}^{128}$ tal que $Enc_k(m_0) = c_0$ con aproximadamente $\sqrt{2^{128}} = 2^{64}$ iteraciones

- Estas iteraciones incluyen evaluar el circuito para AES, comparar y deshacer cálculos

Segunda consecuencia: satisfacibilidad

Sea 3-CNF-SAT el problema de determinar, dada una fórmula proposicional φ en 3-CNF, si φ es satisfacible

El algoritmo usual para resolver 3-CNF-SAT funciona en tiempo $O(2^n \|\varphi\|)$

- φ es una fórmula proposicional en 3-CNF
- n es el número de variables de φ
- $\|\varphi\|$ es el tamaño de φ
 - Esto es el largo de la fórmula φ considerada como un string

Segunda consecuencia: satisfacibilidad

¿En un computador clásico, es posible resolver 3-CNF-SAT en tiempo $O(2^{\delta n} p(\|\varphi\|))$ para algún $\delta \in (0, 1)$ y polinomio p ?

Desde ahora en adelante usamos la notación: $O(2^{\delta n} \text{poly}(\|\varphi\|))$

$\text{poly}(x)$ representa un polinomio
arbitrario cualquiera

Segunda consecuencia: satisfacibilidad

Vamos a presentar un algoritmo para 3-CNF-SAT que funciona en tiempo $O(2^{0.8792n} \text{poly}(\|\varphi\|))$ en un computador clásico

Dada una fórmula φ , en el algoritmo usamos la siguiente notación

Dado un literal ℓ , $\varphi[\ell \mapsto 1]$ es la fórmula proposicional obtenida desde φ de la siguiente forma:

1. Primero se eliminan de φ todas las cláusulas donde aparece ℓ
2. Después se elimina $\bar{\ell}$ desde cada cláusula que menciona $\bar{\ell}$

Segunda consecuencia: satisfacibilidad

Dado un literal ℓ , $\varphi[\ell \mapsto 0]$ es la fórmula proposicional obtenida desde φ de la siguiente forma:

1. Primero se eliminan de φ todas las cláusulas donde aparece $\bar{\ell}$
2. Después se elimina ℓ desde cada cláusula que menciona ℓ

Dados literales $\ell_1, \ell_2, \dots, \ell_k$ y valores $i_1, i_2, \dots, i_k \in \{0, 1\}$:

$$\varphi[\ell_1 \mapsto i_1, \ell_2 \mapsto i_2, \dots, \ell_k \mapsto i_k] = \varphi[\ell_1 \mapsto i_1][\ell_2 \mapsto i_2] \cdots [\ell_k \mapsto i_k]$$

Segunda consecuencia: satisfacibilidad

SAT-Ramificación(φ)

1. **if** φ no tiene cláusulas **then return true**
2. **if** φ tiene una cláusula vacía **then return false**
3. Elija una cláusula $C = (\ell_1 \vee \dots \vee \ell_r)$ en φ con $1 \leq r \leq 3$
4. **for** $i = 1$ **to** r **do**
 1. **if** SAT-Ramificación($\varphi[\ell_1 \mapsto 0, \dots, \ell_{i-1} \mapsto 0, \ell_i \mapsto 1]$) **then return true**
5. **return false**

Segunda consecuencia: satisfacibilidad

Sea $T(n)$ el tiempo de ejecución de **SAT-Ramificación** en el peor caso

Tenemos que:

$$T(n) \leq T(n-1) + T(n-2) + T(n-3) + p(\|\varphi\|)$$

donde p es un polinomio

Solucionando la ecuación obtenemos que **SAT-Ramificación** es $O(2^{0.8792n} \text{poly}(\|\varphi\|))$

- El orden del algoritmo es $O(\alpha^n \text{poly}(\|\varphi\|))$, donde α es la constante de tribonacci, vale decir, la raíz real del polinomio $x^3 = x^2 + x + 1$

Segunda consecuencia: satisfacibilidad

¿Cuánto más podemos reducir δ ?

- Existe un algoritmo de orden $O(2^{0.4092n} \text{poly}(\|\varphi\|))$ para 3-CNF-SAT

Sea s_3 el ínfimo entre todos los valores $\delta \in (0, 1)$ tales que 3-CNF-SAT puede ser resuelto en tiempo $O(2^{\delta n} \text{poly}(\|\varphi\|))$

Exponential time hypothesis (ETH): $s_3 > 0$

- Esto incluye algoritmos aleatorizados con error acotado

Segunda consecuencia: satisfacibilidad

Sea s_k el ínfimo entre todos los valores $\delta \in (0, 1)$ tales que k -CNF-SAT puede ser resuelto en tiempo $O(2^{\delta n} \text{poly}(\|\varphi\|))$

Sabemos que $s_3 \leq s_4 \leq s_5 \leq \dots \leq 1$

El algoritmo **SAT-Ramificación** se puede extender fácilmente al caso de k -CNF-SAT

- ¿Cómo se hace esto?

Segunda consecuencia: satisfacibilidad

¿Cuál es la complejidad de **SAT-Ramificación** para k -CNF-SAT?

- En este caso el tiempo está dado por la ecuación
$$T(n) \leq T(n-1) + \dots + T(n-k) + q(\|\varphi\|),$$
 donde q es un polinomio
- El orden es $O(2^{\delta_k n} \text{poly}(\|\varphi\|))$, donde $\delta_3 < \delta_4 < \delta_5 < \dots < 1$ y $\lim_{k \rightarrow \infty} \delta_k = 1$

Strong exponential time hypothesis (SETH): $\lim_{k \rightarrow \infty} s_k = 1$

- Esto también incluye algoritmos aleatorizados con error acotado

Segunda consecuencia: satisfacibilidad

SETH nos dice que para todo $\delta \in (0, 1)$, existe $k \geq 3$ tal que k -CNF-SAT no puede ser resuelto en tiempo $O(2^{\delta n} \text{poly}(\|\varphi\|))$

- En un computador clásico

Como consecuencia de SETH obtenemos que no existe $\delta \in (0, 1)$ tal que CNF-SAT puede ser resuelto en tiempo $O(2^{\delta n} \text{poly}(\|\varphi\|))$

- Y lo mismo se concluye para SAT

Segunda consecuencia: satisfacibilidad

¿Es SETH cierta en un computador cuántico?

- No, SETH no es cierta por el algoritmo de Grover

Sea φ una fórmula proposicional

- Las variables de φ son $\{x_0, \dots, x_{n-1}\}$
- φ no está necesariamente en CNF

Segunda consecuencia: satisfacibilidad

Sea φ una fórmula proposicional

- Las variables de φ son $\{x_0, \dots, x_{n-1}\}$

Se puede construir un circuito clásico $F : \{0, 1\}^n \rightarrow \{0, 1\}$ tal que:

Una asignación $x_0 \mapsto i_0, \dots, x_{n-1} \mapsto i_{n-1}$ satisface φ
si y solo si $F(i_{n-1}, \dots, i_0) = 1$

Además, se tiene que el número de compuertas de F es $O(\|\varphi\|)$

Segunda consecuencia: satisfacibilidad

Para el circuito F construimos un circuito cuántico U_F tal que para $x \in \{0, 1\}^n$ e $y \in \{0, 1\}$:

$$U_F |y 0^r x\rangle = |(y \oplus F(x)) 0^r x\rangle$$

U_F utiliza $O(\|\varphi\|)$ compuertas y $O(\|\varphi\|)$ qubits auxiliares

Segunda consecuencia: satisfacibilidad

Para $x \in \{0, 1\}^n$, tenemos que $F(x) = 1$ si y solo si la asignación que corresponde a x satisface φ

¿Qué representa entonces U_F ?

- Si suponemos que $x \in \{0, 1\}^n$ es una posición en una qRAM, ¿qué nos indica el oráculo U_F ?

Por supuesto no construimos esta qRAM, la tenemos dada de manera implícita a través del circuito U_F

Segunda consecuencia: satisfacibilidad

Construimos O_F a partir de U_F , y utilizamos el algoritmo de Grover para el oráculo O_F

Podemos descubrir entonces una asignación $x \in \{0, 1\}^n$ que satisface φ en tiempo $O(\sqrt{2^n} \text{poly}(\|\varphi\|)) = O(2^{\frac{n}{2}} \text{poly}(\|\varphi\|))$

- Si la fórmula es satisfacible, el algoritmo tiene una probabilidad de no retornar un resultado
 - Esta probabilidad se reduce ejecutando de nuevo
- Si la fórmula no es satisfacible no se entrega tal valuación